

Дизайн и анализ на алгоритми упражнение 11

Станислав Игов (Стъката)

2025/2026

Срязващи върхове. Мостове. Силносвързани компоненти. Топологично сортиране. Задачи.

Срязващи върхове (Articulation Points / Cut Vertices)

Дефиниция

Нека $G = (V, E)$ е неориентиран свързан граф. Връх $v \in V$ е **срязващ връх (articulation point)**, ако премахването му (заедно с инцидентните му ребра) увеличава броя на свързаните компоненти на графа. С други думи, $G \setminus \{v\}$ е несвързан.

Идея на алгоритъма (Tarjan)

Алгоритъмът се базира на DFS обхождане на графа. По време на DFS-а за всеки връх v поддържа две стойности:

- $disc[v]$ – времето на откриване на v (поредният номер, в който DFS посещава v).
- $low[v]$ – минималното $disc$ стойност, достижима от поддървото на v чрез дървесни ребра и най-много едно обратно ребро (back edge).

Връх u е срязващ, ако е изпълнено поне едно от следните условия:

- **Коренът на DFS дървото:** u е корен и има поне две деца в DFS дървото. Ако премахнем корен с ≥ 2 деца, поддърветата се разпадат.
- **Вътрешен връх:** u не е корен и има дете v в DFS дървото, за което $low[v] \geq disc[u]$. Това означава, че от поддървото на v не може да се стигне до предшественик на u без да се минава през u .

Свойства

- Алгоритъмът работи чрез еднократно DFS обхождане.
- Коренът на DFS дървото е срязващ тогава и само тогава, когато има ≥ 2 деца.
- За некоренов връх u : u е срязващ $\Leftrightarrow \exists$ дете v : $low[v] \geq disc[u]$.
- Лист в DFS дървото никога не е срязващ връх.

Времева сложност

$$Time \in O(|V| + |E|)$$

Алгоритъмът извършва едно DFS обхождане, при което за всяко ребро и всеки връх се извършва константна работа.

Мостове (Bridges)

Дефиниция

Нека $G = (V, E)$ е **неориентиран свързан граф**. Ребро $(u, v) \in E$ е **мост (bridge)**, ако премахването му увеличава броя на свързаните компоненти на графа. С други думи, $G \setminus \{(u, v)\}$ е несвързан.

Идея на алгоритъма

Алгоритъмът за намиране на мостове е много подобен на алгоритъма за срязващи върхове. Отново използваме DFS и поддържаме масиви $disc[]$ и $low[]$ със същите дефиниции.

Ребро (u, v) е мост, ако v е дете на u в DFS дървото и $low[v] > disc[u]$. Това означава, че от поддървото на v няма обратно ребро, което да стига до u или по-нагоре в дървото. Следователно единствената връзка между поддървото на v и останалата част от графа е реброто (u, v) .

Забележка: Разликата с условието за срязващ връх е стриктното неравенство: при мостове е $low[v] > disc[u]$, а при срязващи върхове е $low[v] \geq disc[u]$.

Свойства

- Алгоритъмът работи чрез еднократно DFS обхождане.
- Ребро (u, v) е мост $\Leftrightarrow v$ е дете на u и $low[v] > disc[u]$.
- Ребро, което участва в цикъл, не може да бъде мост.
- Ако граф има мост, то поне един от крайните му върхове е срязващ връх (но не непременно обратното).
- Мост, за който и двата края имат степен ≥ 2 , генерира два срязващи върха.

Времева сложност

$$Time \in O(|V| + |E|)$$

Като при срязващите върхове – едно DFS обхождане.

Силносвързани компоненти (Strongly Connected Components)

Дефиниция

Нека $G = (V, E)$ е **ориентиран граф**. Множество от върхове $C \subseteq V$ е **силносвързана компонента (SCC)**, ако за всеки два върха $u, v \in C$ съществува път от u до v и път от v до u . Силносвързана компонента е максимална, ако не може да бъде разширена с допълнителен връх.

Алгоритъм на Kosaraju (идея)

Алгоритъмът на Kosaraju намира всички SCC чрез две DFS обхождания:

1. **Първо DFS:** Обхождаме целия граф G с DFS. Когато приключим обработката на даден връх (т.е. всички негови съседи са обходени), го добавяме в стек. Така в стека върховете са подредени по времето на завършване (finish time) в намаляващ ред.
2. **Транспониране:** Построяваме транспонирания граф G^T – същият граф, но с обърнати ребра. Ако $(u, v) \in E$, то $(v, u) \in E^T$.
3. **Второ DFS:** Изваждаме върхове от стека (в ред на намаляващ finish time) и пускаме DFS от всеки непосетен връх в G^T . Всички върхове, достигнати от едно такова DFS, формират една SCC.

Алгоритъм на Tarjan (идея)

Алгоритъмът на Tarjan също намира SCC, но с **едно DFS обхождане**. Поддържа стек и масиви $disc[]$ и $low[]$. Връх v е корен на SCC, ако $low[v] = disc[v]$. Когато открием такъв корен, изваждаме от стека всички върхове до v включително – те формират една SCC.

Свойства

- Всеки връх принадлежи на точно една SCC.
- Ако свием всяка SCC в един връх, получаваме **DAG (ориентиран ацикличен граф)** – т.нар. граф на компонентите (condensation graph).
- Граф е силно свързан $\Leftrightarrow$ има точно една SCC.
- Kosaraju използва две DFS обхождания, а Tarjan – едно.
- SCC разделянето е уникално (не зависи от реда на обхождане).

Времева сложност

И двата алгоритъма (Kosaraju и Tarjan) работят в:

$$Time \in O(|V| + |E|)$$

Kosaraju извършва две DFS обхождания и едно транспониране, всяко от които е линейно. Tarjan извършва едно DFS обхождане.

Топологично сортиране (Topological Sort)

Дефиниция

Нека $G = (V, E)$ е **ориентиран ацикличен граф (DAG)**. **Топологично сортиране** е линейна наредба на върховете, така че за всяко ребро $(u, v) \in E$ връхът u е преди v в наредбата.

Защо е полезно

- **Планиране на задачи с зависимости:** Ако задача A трябва да бъде изпълнена преди задача B , топологичното сортиране дава валиден ред на изпълнение.
- **Компилиране на изходен код:** Определяне на реда за компилация на модули с зависимости.
- **Динамично програмиране върху DAG:** Много DP задачи върху DAG-ове изискват обработка на върховете в топологичен ред (напр. най-кратък или най-дълъг път в DAG).
- **Оценяване на формули в електронна таблица:** Клетките с формули се оценяват в ред, съобразен със зависимостите.

Свойства

- Топологично сортиране съществува тогава и само тогава, когато графът е DAG (няма цикли).
- Топологичното сортиране **не е уникално** – може да има повече от една валидна наредба.
- DAG с n върха има поне едно топологично сортиране.
- Всеки DAG има поне един връх с входяща степен 0 (source) и поне един с изходяща степен 0 (sink).

Подход 1: Чрез DFS (използвайки finish time)

Идея

Пускаме DFS върху целия граф. Когато обработката на даден връх приключи (всички съседи са обходени), добавяме го в стек (или в началото на списък). След приключване на DFS, стекът (или списъкът) съдържа върховете в топологичен ред.

Интуицията е следната: ако $(u, v) \in E$, то DFS ще приключи обработката на v преди u (защото v е наследник). Следователно u ще бъде добавен в стека след v , т.е. u ще бъде преди v при извличане.

Псевдокод

```
TopologicalSortDFS(G(V, E))
@1  visited[1..n] <- {false..false}
@2  stack <- empty
@3
@4  function DFS(u)
@5      visited[u] <- true
@6      forall v in adj(u) do
@7          if not visited[v] then
@8              DFS(v)
@9          end if
@10     end for
@11     stack.push(u)
@12 end function
@13
@14 forall v in V do
@15     if not visited[v] then
@16         DFS(v)
@17     end if
@18 end for
@19 return stack.reverse()
```

Времева сложност

$$Time \in O(|V| + |E|)$$

Задача 1: Градинката на стефчо

Условие: Във ФМИ всеки си има градинка с цветя, която представлява редица от няколко различни по вид цветя (видовете са номерирани с целите числа от 1 до m). В градинката не може да има няколко цветя от един и същи вид. Стефчо иска да си направи градинка, спазвайки следните правила:

- В градинката му да има точно по едно цвете от видовете, които приятелите му имат, и да няма други видове цветя.
- Ако цвете x се намира преди цвете y в градинката на някой негов приятел, то и в неговата градинка трябва цвете x да е преди y .

Вход: Число K (брой приятели). За всеки приятел – списък от цветята в неговата градина подредени, в който са подредени в градинката.

Изход: `true`, ако може да се направи такава градинка, `false` иначе.

Идея

Задачата се свежда до топологично сортиране. Всяко ограничение “цвете x е преди цвете y ” е ребро в ориентиран граф. Ако този граф е DAG, градинката може да се направи; ако има цикъл – не може.

Стъпки

1. Намираме множеството от всички видове цветя, срещани в градинките на приятелите – това са върховете на графа.
2. За всеки приятел с градинка $c_1, c_2, \dots, c_t$ добавяме ориентирани ребра (c_i, c_{i+1}) за $i = 1, \dots, t - 1$. Тези ребра кодират ограниченията за наредба.
3. Пускаме алгоритъм за топологично сортиране върху построенния граф.
4. Ако топологичното сортиране завърши успешно (всички върхове са включени в наредбата), отговаряме **true**. Ако не (графът съдържа цикъл), отговаряме **false**.

Коректност

Всяко ребро (c_i, c_{i+1}) от градинката на приятел означава “ c_i трябва да е преди c_{i+1} ”. Достатъчно е да добавяме само ребра между последователни елементи, защото наредбата е транзитивна: от $c_1 \rightarrow c_2$ и $c_2 \rightarrow c_3$ следва $c_1 \rightarrow c_3$ чрез транзитивността на топологичната наредба. Градинката на Стефчо може да бъде построена тогава и само тогава, когато всички ограничения са съвместими, т.е. няма цикъл. Ако има цикъл, то съществуват цветя $x_1, x_2, \dots, x_k$ с ограничения x_1 преди x_2 , x_2 преди x_3 , $\dots$, x_k преди x_1 , което е невъзможно. Ако няма цикъл, топологичното сортиране дава валидна наредба, която удовлетворява всички ограничения едновременно.

Времева сложност

Нека n е общият брой различни видове цветя, а L е сумата от дължините на всички градинки. Построяването на графа е $O(L)$ (за всеки приятел добавяме $O(t_i - 1)$ ребра, а $\sum t_i = L$). Графът има най-много n върха и $O(L)$ ребра. Топологичното сортиране е $O(n + L)$.

$$Time \in O(n + L)$$

Задача 2: Кактус граф (IsCactus)

Условие: Кактус е ориентиран граф $G = (V, E)$, за който няма два различни цикъла $C_1 \neq C_2$, които имат общ връх.

Част 1: Да се докаже, че ориентиран граф G е кактус точно тогава, когато всяка негова компонента на силна свързаност е тривиална (съдържа точно един връх) или броят на ребрата в нея е равен на броя на нейните върхове.

Част 2: Да се предложи алгоритъм с времева сложност $O(|V| + |E|)$ за проблема IsCactus.

Вход: $G = (V, E)$ – ориентиран граф с върхове $V = \{1, 2, \dots, n\}$.

Изход: Да, ако G е кактус; Не, иначе.

Доказателство на Част 1

($\Rightarrow$) Ако G е кактус, то всяка SCC е тривиална или има $|E_{SCC}| = |V_{SCC}|$.

Нека S е нетривиална SCC с k върха ($k \geq 2$). Тъй като S е силно свързана, тя съдържа поне един цикъл. Понеже G е кактус, в S има точно един цикъл (ако имаше два различни, те биха имали общ връх, тъй като S е силно свързана). Един цикъл с k върха има точно k ребра. Ако S имаше допълнително ребро извън цикъла, то заедно с част от цикъла би образувало втори цикъл, което противоречи на свойството за кактус. Следователно $|E_S| = |V_S| = k$.

($\Leftarrow$) Ако всяка SCC е тривиална или има $|E_{SCC}| = |V_{SCC}|$, то G е кактус.

Допускаме противното: G не е кактус, т.е. съществуват два различни цикъла $C_1 \neq C_2$ с общ връх v . Тъй като v лежи и в двата цикъла, всички върхове от C_1 и C_2 са взаимно достижими (чрез обхождане на цикъла), следователно те принадлежат на една SCC. Обединението $C_1 \cup C_2$ съдържа поне едно ребро, което не е в C_1 (тъй като $C_2 \neq C_1$), следователно $|E_S| > |V_S|$ за тази SCC – противоречие с условието.

Идея (Част 2)

Използваме резултата от Част 1: графът е кактус тогава и само тогава, когато всяка SCC е тривиална или има равен брой ребра и върхове.

Стъпки

1. Намираме всички силно свързани компоненти на графа с алгоритъма на Kosaraju или Tarjan.
2. За всяка нетривиална SCC ($s \geq 2$ върха) проверяваме дали броят на ребрата вътре в нея е равен на броя на върховете.
3. Ако за всяка SCC условието е изпълнено, отговаряме “Да”. Ако за някоя SCC $|E_{SCC}| \neq |V_{SCC}|$, отговаряме “Не”.

Коректност

Следва директно от доказателството на Част 1. Алгоритъмът проверява точно необходимото и достатъчно условие за кактус: всяка SCC е тривиална или има $|E_{SCC}| = |V_{SCC}|$.

За броенето на ребрата вътре в дадена SCC: след като намерим SCC и присвоим на всеки връх номер на компонентата му, за всяко ребро (u, v) проверяваме дали $comp[u] = comp[v]$. Ако да, то реброто е вътрешно за тази компонента и увеличаваме броя на ребрата за нея.

Времева сложност

Намирането на SCC е $O(|V| + |E|)$. Преминането по всички ребра за преброяване на вътрешните ребра на всяка SCC е $O(|E|)$. Проверката на условието за всяка компонента е $O(|V|)$.

$$Time \in O(|V| + |E|)$$

Задача 3: Области на дъска (Regions, MoreRegions, OneMoreRegions)

Условие: Дъска $n \times n$, част от чиито полета са маркирани, е представена с матрица $A[1..n][1..n]$ от нули и единици ($A[i][j] = 1$ ако полето е маркирано, 0 иначе). **Област** е максимално по включване множество от клетки, което може да бъде обходено от шахматен цар, без да стъпва на маркирана клетка.

Подзадача 1: Regions

Вход: $A[1..n][1..n]$ – представяне на дъска с маркирани клетки.

Изход: r – брой различни области на дъската.

Идея

Задачата е класическо намиране на свързани компоненти в неявен граф (решетка). Всяка немаркирана клетка е връх, а две клетки са свързани, ако са съседни по хоризонтал, вертикал или диагонал (8-свързаност, тъй като шахматен цар може да стъпва в 8 посоки).

Стъпки

1. Инициализираме брояч $r = 0$.
2. Обхождаме всички клетки (i, j) на дъската. Ако $A[i][j] = 0$ и клетката не е посетена, стартираме BFS (или DFS) от (i, j) , маркирайки всички достижими немаркирани клетки като посетени. Увеличаваме r с 1.

3. При BFS/DFS от дадена клетка разглеждаме всичките 8 съседни клетки (хоризонтално, вертикално и диагонално). Преминуваме само към немаркирани и непосетени съседи.
4. Връщаме r .

Коректност

Всяка област е свързана компонента в графа на 8-свързаност върху немаркираните клетки. BFS/DFS от непосетена клетка обхожда точно една такава компонента. Броят на стартиранията е равен на броя на компонентите, т.е. на броя области.

Времева сложност

Всяка клетка се посещава най-много веднъж. Всяка клетка има най-много 8 съседа. Общо:

$$Time \in O(n^2)$$

Подзадача 2: MoreRegions

Вход: $A[1..n][1..n]$ – представяне на дъска с маркирани клетки.

Изход: $C[1..m][1..2]$, където $(C[i][1], C[i][2])$ са всички клетки, за които ако се маркира и тя, броят на областите ще се увеличи.

Идея

Клетка, чието маркиране увеличава броя на областите, е **срязващ връх** в графа на 8-свързаност върху немаркираните клетки. Маркирането на такава клетка “разцепва” нейната област на две или повече части, което увеличава общия брой области. Маркирането на клетка, която не е срязващ връх, не разцепва областта (просто я намалява с една клетка), така че броят области остава същият или намалява.

Има едно изключение: ако клетката е единствената в своята област (изолирана), маркирането ѝ премахва цяла област и броят намалява. Такива клетки не трябва да се включват в отговора.

Стъпки

1. Построяваме неявен граф на 8-свързаност: върхове са немаркираните клетки, ребра – между 8-съседни немаркирани клетки.
2. Пускаме алгоритъма за намиране на срязващи върхове върху всяка свързана компонента на този граф.
3. Всеки срязващ връх (i, j) добавяме в изходния масив C .

Коректност

Нека (i, j) е немаркирана клетка в област с ≥ 2 клетки. Маркирането на (i, j) премахва този връх от графа. Ако (i, j) е срязващ връх, компонентата се разпада на ≥ 2 части, т.е. една област става ≥ 2 области. Тъй като губим 1 област, но печелим ≥ 2 , общият брой се увеличава. Ако (i, j) не е срязващ връх и не е изолирана, компонентата остава свързана (с една клетка по-малко), броят области не се увеличава. Ако (i, j) е изолирана (единствена в областта си), маркирането ѝ унищожава областта, но тя не е срязващ връх (няма деца в DFS дърво), така че алгоритъмът коректно я пропуска.

Времева сложност

Алгоритъмът за срязващи върхове е $O(|V| + |E|)$. Тук $|V| \leq n^2$ и $|E| \leq 4n^2$ (всяка клетка има ≤ 8 съседа, но всяко ребро се брой веднъж):

$$Time \in O(n^2)$$

Подзадача 3: OneMoreRegions

Вход: $A[1..n][1..n]$ – представяне на дъска с маркирани клетки.

Изход: $C[1..m][1..2]$, където $(C[i][1], C[i][2])$ са всички клетки, за които ако се маркира и тя, броят на областите ще се увеличи с **точно 1**.

Идея

Клетка увеличава броя на областите с точно 1, ако маркирането ѝ разделя нейната компонента на точно 2 части. Това съответства на срязващ връх, чието премахване разделя компонентата на точно 2 свързани части.

При алгоритъма за срязващи върхове с DFS за всеки срязващ връх u можем да преброим на колко части се разпада компонентата при премахването му. Броят нови компоненти, получени от премахването на u , е:

- Ако u е корен на DFS дървото: броят деца в DFS дървото.
- Ако u не е корен: $1 + \text{брой деца } v, \text{ за които } low[v] \geq disc[u]$.

Стъпки

1. Построяваме графа на 8-свързаност върху немаркираните клетки, както в подзадача 1.
2. Пускаме алгоритъма за срязващи върхове върху всяка свързана компонента. За всеки срязващ връх u пресмятаме $parts(u)$ – броя компоненти, на които се разпада областта при премахване на u :
 - Ако u е корен: $parts(u) = \text{брой деца на } u \text{ в DFS дървото}$.
 - Ако u не е корен: $parts(u) = 1 + |\{v : v \text{ е дете на } u \text{ и } low[v] \geq disc[u]\}|$.
3. Клетката u влиза в отговора тогава и само тогава, когато $parts(u) = 2$. Това означава, че маркирането на u разделя областта на точно 2 части, т.е. увеличава общия брой области с точно 1.
4. Добавяме в изходния масив C всички клетки u , за които $parts(u) = 2$.

Коректност

Нека u е срязващ връх в компонента S . При премахване на u компонентата S се разпада на $parts(u)$ части. Преди маркирането имаме 1 област от S , след маркирането имаме $parts(u)$ области. Увеличението е $parts(u) - 1$. Искаме увеличение точно 1, следователно $parts(u) = 2$.

За коренов връх с k деца: премахването му дава k компоненти, т.е. увеличение $k - 1$. За $k = 2$ получаваме увеличение 1.

За некоренов връх u : при премахването му всяко дете v с $low[v] \geq disc[u]$ поражда отделна компонента (поддървото на v не може да достигне нагоре без u). Останалите деца и родителят на u остават в една обща компонента. Следователно общо $parts(u) = 1 + |\{v : low[v] \geq disc[u]\}|$. За увеличение 1 ни трябва точно едно такова дете v .

Времева сложност

Същата като подзадача 2 – еднократно DFS обхождане с допълнително броене:

$$Time \in O(n^2)$$